

Before your chosen LLM can act on your data, you first need to process the data and load it. This has parallels to data cleaning/feature engineering pipelines in the ML world, or ETL pipelines in the traditional data setting.

This ingestion pipeline typically consists of three main stages:

1. Load the data
2. Transform the data
3. Index and store the data

We cover indexing/storage in [future sections](#). In this guide we'll mostly talk about loaders and transformations.

Loaders

Before your chosen LLM can act on your data you need to load it. The way LlamaIndex does this is via data connectors, also called `Reader`. Data connectors ingest data from different data sources and format the data into `Document` objects. A `Document` is a collection of data (currently text, and in future, images and audio) and metadata about that data.

Loading using SimpleDirectoryReader

The easiest reader to use is our SimpleDirectoryReader, which creates documents out of every file in a given directory. It is built in to LlamaIndex and can read a variety of formats including Markdown, PDFs, Word documents, PowerPoint decks, images, audio and video.

```
from llama_index.core import SimpleDirectoryReader

documents = SimpleDirectoryReader("./data").load_data()
```

Using Readers from LlamaHub

Because there are so many possible places to get data, they are not all built-in. Instead, you download them from our registry of data connectors, [LlamaHub](#).

In this example LlamaIndex downloads and installs the connector called [DatabaseReader](#), which runs a query against a SQL database and returns every row of the results as a `Document` :

```
from llama_index.core import download_loader

from llama_index.readers.database import DatabaseReader

reader = DatabaseReader(
    scheme=os.getenv("DB_SCHEME"),
    host=os.getenv("DB_HOST"),
    port=os.getenv("DB_PORT"),
    user=os.getenv("DB_USER"),
    password=os.getenv("DB_PASS"),
    dbname=os.getenv("DB_NAME"),
)

query = "SELECT * FROM users"
documents = reader.load_data(query=query)
```

There are hundreds of connectors to use on [LlamaHub](#)!

Creating Documents directly

Instead of using a loader, you can also use a Document directly.

```
from llama_index.core import Document

doc = Document(text="text")
```

Transformations

After the data is loaded, you then need to process and transform your data before putting it into a storage system. These transformations include chunking, extracting metadata, and embedding each chunk. This is necessary to make sure that the data can be retrieved, and used optimally by the LLM.

Transformation input/outputs are `Node` objects (a `Document` is a subclass of a `Node`). Transformations can also be stacked and reordered.

We have both a high-level and lower-level API for transforming documents.

High-Level Transformation API

Indexes have a `.from_documents()` method which accepts an array of Document objects and will correctly parse and chunk them up. However, sometimes you will want greater control over how your documents are split up.

```
from llama_index.core import VectorStoreIndex

vector_index = VectorStoreIndex.from_documents(documents)
vector_index.as_query_engine()
```

Under the hood, this splits your Document into Node objects, which are similar to Documents (they contain text and metadata) but have a relationship to their parent Document.

If you want to customize core components, like the text splitter, through this abstraction you can pass in a custom `transformations` list or apply to the global `Settings` :

```
from llama_index.core.node_parser import SentenceSplitter

text_splitter = SentenceSplitter(chunk_size=512, chunk_overlap=10)

# global
from llama_index.core import Settings

Settings.text_splitter = text_splitter

# per-index
index = VectorStoreIndex.from_documents(
    documents, transformations=[text_splitter]
)
```

Lower-Level Transformation API

You can also define these steps explicitly.

You can do this by either using our transformation modules (text splitters, metadata extractors, etc.) as standalone components, or compose them in our declarative [Transformation Pipeline interface](#).

Let's walk through the steps below.

Splitting Your Documents into Nodes

A key step to process your documents is to split them into “chunks”/Node objects. The key idea is to process your data into bite-sized pieces that can be retrieved / fed to the LLM.

LlamaIndex has support for a wide range of [text splitters](#), ranging from paragraph/sentence/token based splitters to file-based splitters like HTML, JSON.

These can be [used on their own or as part of an ingestion pipeline](#).

```
from llama_index.core import SimpleDirectoryReader
from llama_index.core.ingestion import IngestionPipeline
from llama_index.core.node_parser import TokenTextSplitter

documents = SimpleDirectoryReader("./data").load_data()

pipeline = IngestionPipeline(transformations=[TokenTextSplitter(), ...])

nodes = pipeline.run(documents=documents)
```

Adding Metadata

You can also choose to add metadata to your documents and nodes. This can be done either manually or with [automatic metadata extractors](#).

Here are guides on 1) [how to customize Documents](#), and 2) [how to customize Nodes](#).

```
document = Document(
    text="text",
    metadata={"filename": "<doc_file_name>", "category": "<category>"},
)
```

Adding Embeddings

To insert a node into a vector index, it should have an embedding. See our [ingestion pipeline](#) or our [embeddings guide](#) for more details.

Creating and passing Nodes directly

If you want to, you can create nodes directly and pass a list of Nodes directly to an indexer:

```
from llama_index.core.schema import TextNode

node1 = TextNode(text="<text_chunk>", id_="<node_id>")
node2 = TextNode(text="<text_chunk>", id_="<node_id>")

index = VectorStoreIndex([node1, node2])
```